CMPE 220 · SYSTEM SOFTWARE · SAN JOSÉ STATE UNIVERSITY · FALL 2026

Program Layout & Execution

Project Part 2 · Team 13 · the T13 16-bit CPU

Manoj Konda	`kmanoj03`
Sai Karan Akula	`karanAkula`
Yashashav Devalapalli Kamalraj	`yashashav-dk`

Code: `https://github.com/yashashav-dk/cmpe220-team13`
Built Nov 07, 2026 from the code by `make report`

How to run

Step	Command (repo root)	Expected
1. Get the code	`git clone https://github.com/yashashav-dk/cmpe220-team13.git` `cd cmpe220-team13`	needs a C99 compiler and make (macOS clang or Linux gcc)
2. Build, test, run every program	`make check`	last test line: `programs: all passed`
3. Assemble and run	`build/t13as programs/recfact.s -o recfact.img` `build/t13sim recfact.img`	`recfact.img: HALT after 112 steps`
4. Loaded executable, decoded	`build/t13view recfact.trc --loaded`	§3
5. Four segments per call	`build/t13view recfact.trc --segments`	12 snapshots (§4)
6. Result	`build/t13view recfact.trc --peek '*result'` `cc programs/recfact.c -o rf && ./rf`	both print `*result=120`

Table 1. how to download and run the code.

1 · Program layout

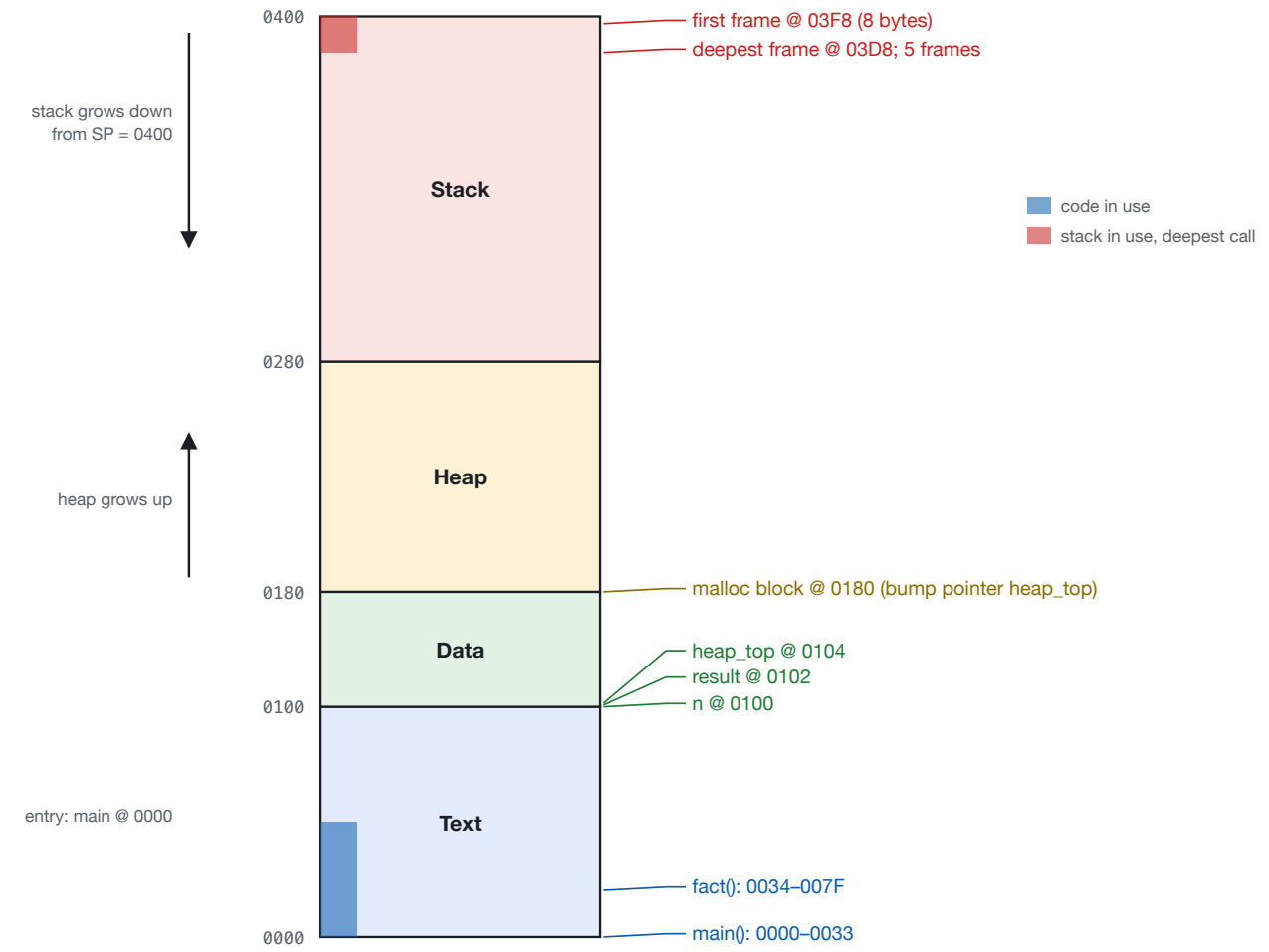


Figure 1. layout of `recfact` in the 1 KiB memory, to scale: Text, Data, Heap, Stack.

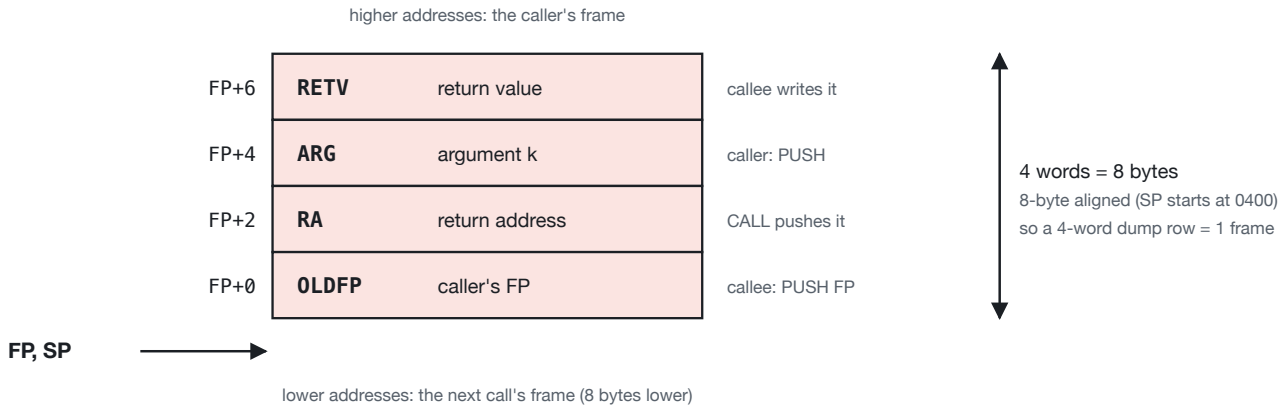


Figure 2. one stack frame (`docs/abi.md`).

2 · Calling sequence

#	Caller	Callee	SP after (first call)
1	SUB SP, #2 return-value slot		03FE
2	PUSH R1 argument		03FC
3	CALL fact pushes return address		03FA
4		PUSH FP · MOV FP, SP prologue	03F8
5		body; MOV [FP+RETV], R0	03F8
6		MOV SP, FP · POP FP · RET epilogue	03FC
7	ADD SP, #2 drop argument		03FE
8	POP R0 the result		0400

Table 2. call sequence; PUSH/POP are SUB/ADD SP, #2 plus a move.

C (programs/*.c)	Addr	Words	T13 assembly (programs/*.s)	Comment
int main(void) {				
result = malloc(sizeof *result);	0000	1610 0104	main: MOV R1, [heap_top]	malloc(2), bump allocator:
	0004	1710 0102	MOV [result], R1	the block starts at heap_top
	0008	2110 0002	ADD R1, #2	
	000C	1710 0104	MOV [heap_top], R1	heap_top += 2
*result = fact(n);	0010	31E0 0002	SUB SP, #2	reserve the return-value slot
	0014	1610 0100	MOV R1, [n]	
	0018	31E0 0002 14E1	PUSH R1	argument k = n
	001E	C000 0034	CALL fact	
	0022	21E0 0002	ADD SP, #2	drop the argument
	0026	120E 21E0 0002	POP R0	R0 = fact(n)
	002C	1610 0102	MOV R1, [result]	
	0030	1410	MOV [R1], R0	store through the pointer, into the Heap
printf(...); (not translated: no I/O)				
return 0;	0032	0000	HALT	
}				
uint16_t fact(uint16_t k) {	0034	31E0 0002 14ED	fact: PUSH FP	prologue: save the caller's FP
	003A	10DE	MOV FP, SP	FP -> this frame
if (k <= 1)	003C	131D 0004	MOV R1, [FP+ARG]	
	0040	A110 0001	CMP R1, #1	
	0044	B500 0050	JGT recurse	
return 1;	0048	1100 0001	MOV R0, #1	
	004C	B000 0072	JMP return	
return k * fact(k - 1);	0050	3110 0001	recurse: SUB R1, #1	k - 1
	0054	31E0 0002	SUB SP, #2	reserve the return-value slot
	0058	31E0 0002 14E1	PUSH R1	argument k - 1
	005E	C000 0034	CALL fact	
	0062	21E0 0002	ADD SP, #2	drop the argument
	0066	120E 21E0 0002	POP R0	R0 = fact(k - 1)
	006C	131D 0004	MOV R1, [FP+ARG]	re-read k (R1 is scratch)
	0070	4001	MUL R0, R1	k * fact(k - 1)
	0072	15D0 0006	return: MOV [FP+RETV], R0	result into the caller's slot
	0076	10ED	MOV SP, FP	epilogue
	0078	12DE 21E0 0002	POP FP	
	007E	D000	RET	
}				
(Data segment)	0100	0005	n: .word 5	
(Data segment)	0102	0000	result: .word 0	the pointer malloc returned
(Data segment)	0104	0180	heap_top:.word 0x0180	first free Heap byte

Table 3. programs/recfact.c next to programs/recfact.s: main and the recursive fact.

3 · Executable loaded into memory

Seg	Addr	Words	Label	Decoded
TEXT	0000	1610 0104	main	MOV R1, [0x0104] → heap_top
	0004	1710 0102		MOV [0x0102], R1 → result
	0008	2110 0002		ADD R1, #2
	000C	1710 0104		MOV [0x0104], R1 → heap_top
	0010	31E0 0002		SUB SP, #2
	0014	1610 0100		MOV R1, [0x0100] → n
	0018	31E0 0002		SUB SP, #2
	001C	14E1		MOV [SP], R1
	001E	C000 0034		CALL 0x0034 → fact
	0022	21E0 0002		ADD SP, #2
	0026	120E		MOV R0, [SP]
	0028	21E0 0002		ADD SP, #2
	002C	1610 0102		MOV R1, [0x0102] → result
	0030	1410		MOV [R1], R0
	0032	0000		HALT
	0034	31E0 0002	fact	SUB SP, #2
	0038	14ED		MOV [SP], FP
	003A	10DE		MOV FP, SP
	003C	131D 0004		MOV R1, [FP+4]
	0040	A110 0001		CMP R1, #1
	0044	B500 0050		JGT 0x0050 → recurse
	0048	1100 0001		MOV R0, #1
	004C	B000 0072		JMP 0x0072 → return
	0050	3110 0001	recurse	SUB R1, #1
	0054	31E0 0002		SUB SP, #2
	0058	31E0 0002		SUB SP, #2
	005C	14E1		MOV [SP], R1
	005E	C000 0034		CALL 0x0034 → fact
	0062	21E0 0002		ADD SP, #2
	0066	120E		MOV R0, [SP]
	0068	21E0 0002		ADD SP, #2
	006C	131D 0004		MOV R1, [FP+4]
	0070	4001		MUL R0, R1
	0072	15D0 0006	return	MOV [FP+6], R0
	0076	10ED		MOV SP, FP
	0078	12DE		MOV FP, [SP]
	007A	21E0 0002		ADD SP, #2
	007E	D000		RET
DATA	0100	0005	n	.word 5
	0102	0000	result	.word 0
	0104	0180	heap_top	.word 384
HEAP	0180–027F	all 0 at load		
STACK	0280–03FF	all 0 at load; SP = 0400		

Table 4. `recfact.s`: executable as loaded into memory, decoded (ENTRY 0000, PC = ENTRY, SP = 0400).

4 · The four segments at every call, every return, and the end

Registers	
PC	0000
R0	0000
R1	0000
R2	0000
R3	0000
FP	0000
SP	0400

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0000
	0104	heap_top	0180
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0000	0000	0000	free (below SP)
03F0	0000	0000	0000	0000	free (below SP)
03E8	0000	0000	0000	0000	free (below SP)
03E0	0000	0000	0000	0000	free (below SP)
03D8	0000	0000	0000	0000	free (below SP)

Figure 3. `recfact.s`: snapshot 0, after loading: Text / Data / Heap / Stack.

Registers	
PC	0034
R0	0000
R1	**0005**
R2	0000
R3	0000
FP	0000
SP	**03FA**

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	**0180**
	0104	heap_top	**0182**
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	**0022** main+22	**0005**	0000	← SP=03FA
03F0	0000	0000	0000	0000	free (below SP)
03E8	0000	0000	0000	0000	free (below SP)
03E0	0000	0000	0000	0000	free (below SP)
03D8	0000	0000	0000	0000	free (below SP)

Figure 4. `recfact.s`: snapshot 1, after step 9 `CALL fact`: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0034
R0	0000
R1	**0004**
R2	0000
R3	0000
FP	**03F8**
SP	**03F2**

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0000	← FP
03F0	0000	**0062** fact+2E	**0004**	0000	← SP=03F2
03E8	0000	0000	0000	0000	free (below SP)
03E0	0000	0000	0000	0000	free (below SP)
03D8	0000	0000	0000	0000	free (below SP)

Figure 5. `recfact.s`: snapshot 2, after step 20 `CALL fact`: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0034
R0	0000
R1	**0003**
R2	0000
R3	0000
FP	**03F0**
SP	**03EA**

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0000	
03F0	**03F8**	0062 fact+2E	0004	0000	← FP
03E8	0000	**0062** fact+2E	**0003**	0000	← SP=03EA
03E0	0000	0000	0000	0000	free (below SP)
03D8	0000	0000	0000	0000	free (below SP)

Figure 6. `recfact.s`: snapshot 3, after step 31 `CALL fact`: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0034
R0	0000
R1	**0002**
R2	0000
R3	0000
FP	**03E8**
SP	**03E2**

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0000	
03F0	03F8	0062 fact+2E	0004	0000	
03E8	**03F0**	0062 fact+2E	0003	0000	← FP
03E0	0000	**0062** fact+2E	**0002**	0000	← SP=03E2
03D8	0000	0000	0000	0000	free (below SP)

Figure 7. `recfact.s`: snapshot 4, after step 42 `CALL fact`: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0034
R0	0000
R1	0001
R2	0000
R3	0000
FP	03E0
SP	03DA

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0000	
03F0	03F8	0062 fact+2E	0004	0000	
03E8	03F0	0062 fact+2E	0003	0000	
03E0	03E8	0062 fact+2E	0002	0000	← FP
03D8	0000	0062 fact+2E	0001	0000	← SP=03DA

Figure 8. `recfact.s`: snapshot 5, after step 53 CALL `fact`: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0062
R0	0001
R1	0001
R2	0000
R3	0000
FP	03E0
SP	03DC

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0000	
03F0	03F8	0062 fact+2E	0004	0000	
03E8	03F0	0062 fact+2E	0003	0000	
03E0	03E8	0062 fact+2E	0002	0000	← FP
03D8	03E0	0062 fact+2E	0001	0001	← SP=03DC

Figure 9. `recfact.s`: snapshot 6, after step 66 RET: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0062
R0	0002
R1	0002
R2	0000
R3	0000
FP	03E8
SP	03E4

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0000	
03F0	03F8	0062 fact+2E	0004	0000	
03E8	03F0	0062 fact+2E	0003	0000	← FP
03E0	03E8	0062 fact+2E	0002	0002	← SP=03E4
03D8	03E0	0062 fact+2E	0001	0001	free (below SP)

Figure 10. `recfact.s`: snapshot 7, after step 76 RET: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0062
R0	0006
R1	0003
R2	0000
R3	0000
FP	03F0
SP	03EC

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0000	
03F0	03F8	0062 fact+2E	0004	0000	← FP
03E8	03F0	0062 fact+2E	0003	0006	← SP=03EC
03E0	03E8	0062 fact+2E	0002	0002	free (below SP)
03D8	03E0	0062 fact+2E	0001	0001	free (below SP)

Figure 11. `recfact.s`: snapshot 8, after step 86 RET: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0062
R0	0018
R1	0004
R2	0000
R3	0000
FP	03F8
SP	03F4

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0000	← FP
03F0	03F8	0062 fact+2E	0004	0018	← SP=03F4
03E8	03F0	0062 fact+2E	0003	0006	free (below SP)
03E0	03E8	0062 fact+2E	0002	0002	free (below SP)
03D8	03E0	0062 fact+2E	0001	0001	free (below SP)

Figure 12. `recfact.s`: snapshot 9, after step 96 RET: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0022
R0	0078
R1	0005
R2	0000
R3	0000
FP	0000
SP	03FC

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0000

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0078	← SP=03FC
03F0	03F8	0062 fact+2E	0004	0018	free (below SP)
03E8	03F0	0062 fact+2E	0003	0006	free (below SP)
03E0	03E8	0062 fact+2E	0002	0002	free (below SP)
03D8	03E0	0062 fact+2E	0001	0001	free (below SP)

Figure 13. `recfact.s`: snapshot 10, after step 106 RET: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

Registers	
PC	0034
R0	0078
R1	0180
R2	0000
R3	0000
FP	0000
SP	0400

Seg	Addr	Label	Value
Text	0000–007F	64 words of code	
Data	0100	n	0005
	0102	result	0180
	0104	heap_top	0182
Heap	0180		0078

Stack	+0 OLDFP	+2 RA	+4 ARG	+6 RETV	
03F8	0000	0022 main+22	0005	0078	free (below SP)
03F0	03F8	0062 fact+2E	0004	0018	free (below SP)
03E8	03F0	0062 fact+2E	0003	0006	free (below SP)
03E0	03E8	0062 fact+2E	0002	0002	free (below SP)
03D8	03E0	0062 fact+2E	0001	0001	free (below SP)

Figure 14. `recfact.s`: snapshot 11, after step 112 HALT: Text / Data / Heap / Stack; shaded = changed since the previous snapshot.

5 · Tests and contributions

Suite	Checks	Result (this build)
`test_isa`	codec: encode / decode / disassemble every instruction	`561 checks, 0 failed`
`test_asm`	assembler parser, symbols, instruction selection, disasm → asm round trip	`773 checks, 0 failed`
`test_assemble`	two-pass assembler, directives, errors, image and listing writers	`158 checks, 0 failed`
`test_sim`	simulator: loader errors, every ALU flag rule, jumps, CALL/RET, faults, trace	`284 checks, 0 failed`
`test_view`	viewer: trace checks (rule 11), every view	`120 checks, 0 failed`
`tests/golden.sh`	t13as images and listings, t13sim traces vs hand-written golden traces, CLI exit codes	`golden: all passed`

`tests/programs.sh`	every program end to end; each .c compiled natively prints what t13sim leaves in memory	`ok power.s: result=64 after 17 steps`
		`ok fact.s: result=40320 after 57 steps`
		`ok fact.c: native cc prints result=40320`
		`ok recfact.s: *result=120 after 112 steps`
		`ok recfact.c: native cc prints *result=120`
		`programs: all passed`

Table 5. test results, captured while building this PDF (`make test`).

Member	Code and documents	Part 1 report	Part 2 report
Manoj Konda	Trace format `docs/trace.md` and the hand-written golden traces `halt.trc` and `power.trc` (the simulator's acceptance tests); CI on Linux gcc/clang and macOS clang (PR #3). Repository layout and `Makefile`, viewer `t13view`, the recursive Part 2 program `recfact`.	§3 and §4 memory tables and their legend, the appendix; title page and page-number template.	§2 `recfact` C next to assembly; §4 four-segment view with stack frames, one snapshot per call and return.
Sai Karan Akula	Reviewed and approved ISA v1.0 (Nov 3). Image format `docs/image.md`, stack-frame layout `docs/abi.md`, simulator `t13sim` (loader and CPU), Program B: 8! from C to T13 assembly.	§1 CPU diagram; §4 Program B; put the Part 1 PDF together.	§1 program-layout and stack-frame diagrams; §2 calling sequence; §4 CALL/RET snapshots and stack checks.
Yashashav Devalapalli Kamalraj	Repository and MIT license. ISA v1.0 `docs/isa.md`, opcode table `isa_ops.def`, codec `isa.c` with 561 checks; assembler `t13as` with listing and manual `docs/asm.md`, 931 checks and golden files; Program A: 8^2 in T13 assembly.	§2 instruction encoding table; §3 Program A.	§3 executable loaded into memory; put the Part 2 PDF together.

Table 6. contributions of each member.